

amore: Simulation and Case-Control Inference for Relational Event Models

Francisco Richter Mendoza
Università della Svizzera italiana, Lugano
richtf@usi.ch

May 2026

Abstract

amore is an R package for the end-to-end relational event modelling workflow: simulation, case-control sampling, endogenous feature engineering, model selection, and inference. It implements a Gillespie-style event simulator with composable exogenous, endogenous, and time-varying global covariate machinery, and exposes a single 41-statistic endogenous catalogue — spanning the full continuous / ordered / exp-decay / interrupted family from Juozaitienė and Wit (2025b) — through both the simulator’s inner loop and a parallel post-hoc feature engine. Every shared statistic is cross-validated row-by-row by a dedicated parity suite. The package ships three real-world REM datasets directly (Classroom, Social Evolution, Manufacturing), so the documented workflows run on real data out of the box. A one-call `compare_models()` helper fits competing endogenous specifications via no-intercept binomial GLM (matched 1:1) or conditional logistic regression (matched 1:k, via `survival::coxph`), and emits a tidy AIC ranking. This whitepaper describes the underlying model, the algorithms, the package’s function surface with worked code listings for every capability, the bundled datasets, the validation experiments (linear recovery, non-linear smooth recovery, time-varying global rate switching, composition), and a real-data analysis on Classroom and Manufacturing including smooth time-effect curves that mirror Figure 6 of Juozaitienė and Wit (2025b). All numbers in the document are produced by reproducibility scripts under `paper/experiments/`.

1 Motivation

Relational event models (REMs) (Butts, 2008; Perry and Wolfe, 2013) treat directed dyadic interactions as a marked point process: each event (s, r, t) is a sender–receiver–time triple, and the intensity at which a given dyad fires is a function of structural covariates, exogenous attributes, and the realized event history. The framework has become a standard tool in social, organizational, and citation network analysis (Vu et al., 2017; Stadtfeld et al., 2017).

Applied REM work has two recurring pain points. First, the *simulator* side is typically bespoke: there is no easy way to mix exogenous actor covariates, endogenous mechanisms (reciprocity, transitivity), and time-varying global covariates (weekday/weekend, policy regimes) into a single generative engine while preserving correct time-inhomogeneous semantics. Second, the *inference* side is fragmented: estimating an REM via partial likelihood (Perry and Wolfe, 2013) requires case-control output (each event paired with one or more unrealized dyads) with the relevant covariate values evaluated at the event time, which most simulators do not produce out of the box.

amore addresses both: it offers a composable simulator that covers all three covariate families, and it emits case-control data in the exact shape consumed by `survival::clogit` and `mgcv::gam`. The package is designed for prototyping new REM methodology, not for high-throughput production runs; its scope and limits are stated explicitly in Section 10.

2 Background

Setup. Let $\mathcal{A}$ be a finite set of actors and let $\mathcal{D} = \{(s, r) : s, r \in \mathcal{A}, s \neq r\}$ be the set of admissible directed dyads (self-loops can optionally be included). An event log is a sequence $(s_1, r_1, t_1), (s_2, r_2, t_2), \dots$ with t_i strictly increasing. We denote by $\mathcal{H}_t$ the realized history up to time t^- .

Intensity. For dyad $d = (s, r)$ and time t , the conditional intensity is

$$\lambda_d(t | \mathcal{H}_t) = \lambda_0(t) \exp\{\eta_d(t; \mathcal{H}_t)\}, \quad \eta_d(t; \mathcal{H}_t) = x_d^\top \beta_x + z_d(t; \mathcal{H}_t)^\top \beta_z + g(t)^\top \beta_g, \quad (1)$$

where $\lambda_0(t)$ is a baseline hazard (constant in this package), x_d are static dyadic / actor covariates, $z_d(t; \mathcal{H}_t)$ are endogenous statistics computed from the past, and $g(t)$ is a vector of global (dyad-independent) covariates that may be piecewise constant in time.

Three covariate families. The three families place different demands on the simulator. *Static actor covariates* only affect the linear predictor once, so they fold into a precomputed $S \times R$ matrix of log weights. *Endogenous statistics* change after *every* realized event and must be recomputed on the fly. *Global time-varying covariates* change at known interval boundaries, not at event times; under standard Gillespie the rate is no longer constant between events, so the algorithm must explicitly track interval boundaries (Section 5).

Case-control likelihood. Conditional on an event (s_i, r_i, t_i) having fired, the probability that dyad d is the one observed is the multinomial

$$\Pr(d | \text{event at } t_i, \mathcal{H}_{t_i}) = \frac{\exp\{\eta_d(t_i; \mathcal{H}_{t_i})\}}{\sum_{d' \in \mathcal{D}} \exp\{\eta_{d'}(t_i; \mathcal{H}_{t_i})\}}. \quad (2)$$

The baseline $\lambda_0(t)$ and the global multiplier $\exp\{g(t)^\top \beta_g\}$ cancel from numerator and denominator; they are not identifiable from (2). Inference on the global effect requires the full likelihood and is not supported in this release (see Section 10). The cancellation is exactly why case-control sampling with one or a few controls per event is computationally attractive: only the contribution of the realized dyad and a small Monte Carlo sample of unrealized dyads need to be evaluated.

3 Bundled datasets

amore ships three of the five real-world REM datasets analysed by Juozaitienė and Wit (2025b) directly, both as tidy event tables in `data/` (loadable via `data(...)`) and as their original raw sources in `inst/extdata/`. The fourth (Enron) is intentionally not bundled because the only publicly archived version is an aggregated daily edge-weight table, not the event-level slice the paper analyses. Table 1 summarises them and Figure 1 shows their basic shape.

Dataset	Event table	Events	Actors	Source
Classroom	<code>classroom_events</code>	691	20	McFarland, 2001
Social Evolution	<code>social_evolution_calls</code>	439	54	Madan et al., 2011
Manufacturing	<code>radoslaw_email</code>	82,927	167	Michalski et al., 2014; Rossi and Ahmed, 2015

Table 1: Bundled REM datasets. Classroom comes from the `networkDynamic` CRAN package, Social Evolution from the `goldfish` GitHub package, and Manufacturing from Network Repository (`ia-radoslaw-email`). Times are normalised to minutes (Classroom) or days since the first event; original Unix epochs are preserved as a `unix_origin` attribute on each event frame.

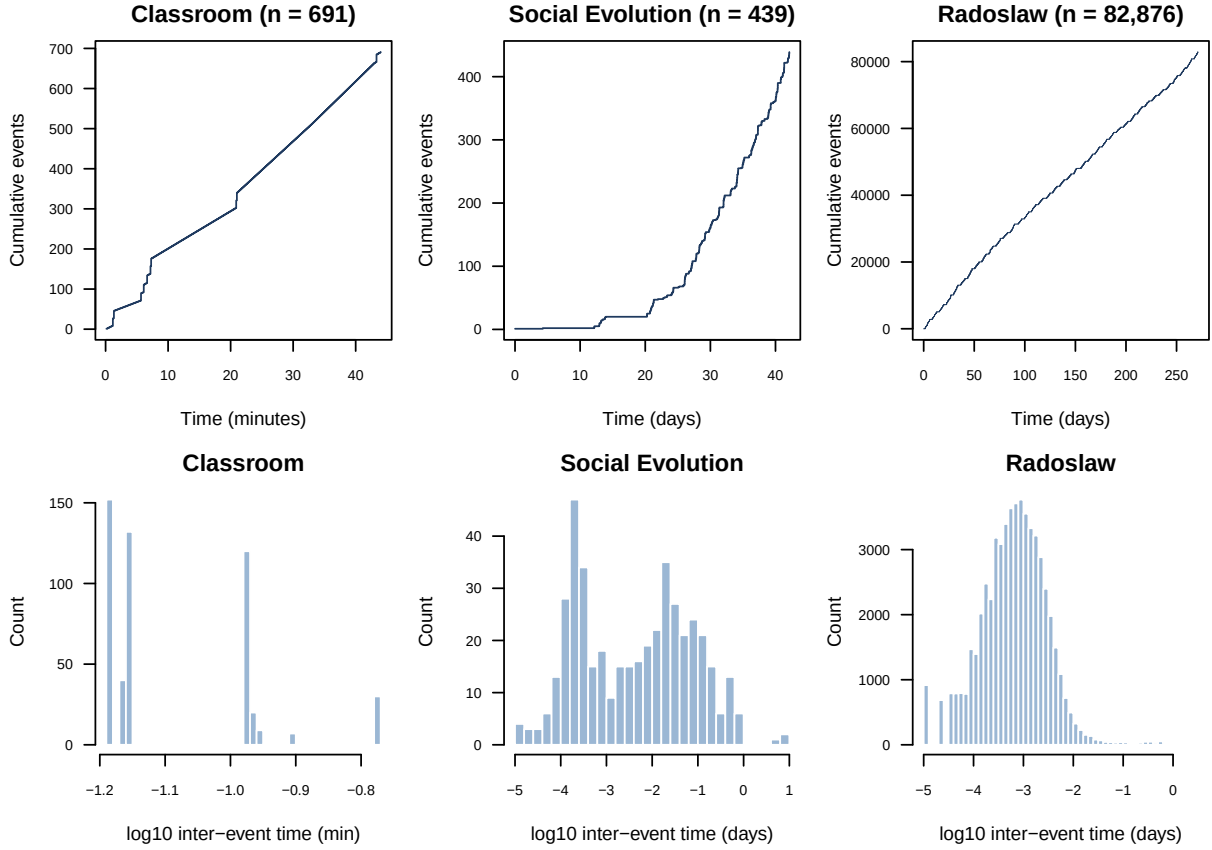


Figure 1: Top row: cumulative event counts. Classroom unfolds in 44 minutes of sustained activity; Social Evolution spans 42 days with intermittent calls; Manufacturing is a long, dense stream over 9 months. Bottom row: histograms of $\log_{10}$ inter-event times. All three exhibit the heavy-tailed inter-event distributions typical of human communication processes.

Classroom (McFarland 2001). Twenty actors (one instructor, nineteen grade-11 / grade-12 students) and 691 directed interactions in 44 minutes of classroom time, with each event tagged by `interaction_type` (social, sanction, task). The actor-covariate table `classroom_actors` carries `sex` (F/M) and `role` (instructor / grade_11 / grade_12). Figure 2 shows the full sociogram.

Social Evolution (Madan et al. 2011). Phone-call events between 54 active students from a larger pool of 84, gathered as part of an MIT residence-hall study. The companion table `social_evolution_actors` provides dorm-floor (`floor`) and grade-type (`gradeType`) actor covariates, and `social_evolution_friendship` records self-reported friendship-survey ties as a parallel event stream useful as a time-varying covariate.

Manufacturing emails (Michalski et al. 2014). A nine-month email log inside a mid-sized manufacturing company (167 active employees, 82,927 directed emails). The dataset exhibits a clear weekday rhythm (visible in the early-day oscillation of the cumulative-events plot, top-right of Figure 1) and a heavy core/periphery activity structure (Figure 3).

4 A capability tour

This section walks through the `amore` function surface with short worked R blocks, in the order a typical study would use them. Section 5 describes the underlying algorithms; this section is the user’s-eye view.

Classroom: 691 directed interactions, 20 actors

McFarland (2001) – edges scaled by event count

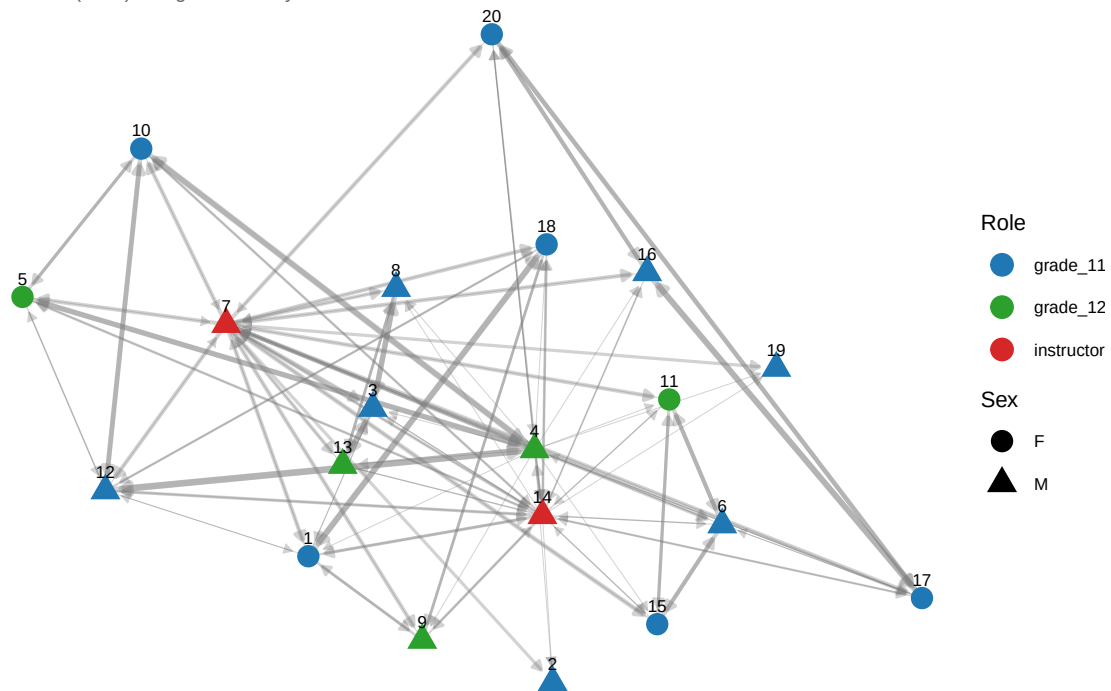


Figure 2: Classroom sociogram (stress layout). Nodes are coloured by role and shaped by sex; directed edges are scaled by event count. The instructor (7, red triangle) is the dominant hub; within-student interactions concentrate on a few high-activity dyads.

4.1 Simulating a relational event stream

```
library(amore)

set.seed(2026)
ev <- simulate_relational_events(
  n_events = 500,
  senders = LETTERS[1:10],
  receivers = LETTERS[1:10],
  baseline_rate = 1,
  allow_loops = FALSE)
head(ev, 3)
#> sender receiver time
#> 1 F A 0.01187...
#> 2 C I 0.02541...
#> 3 G H 0.04318...
```

The minimal call generates a memoryless Gillespie stream. Three optional ingredients make the simulator interesting:

Exogenous covariates. Pass an $S \times R$ `contribution_logits` matrix encoding the linear predictor contribution of each (s, r) dyad. The `simulate_actor_covariates()` helper draws static or AR(1)-dynamic Gaussian traits, and `attach_static_covariates()` joins them back into an event log:

```
covs <- simulate_actor_covariates(
  senders = LETTERS[1:10],
  receivers = LETTERS[1:10],
  covariate_names = c("activity", "popularity"),
```

Radoslaw 30-day slice: $\log(1 + \text{count})$ of (sender, receiver)

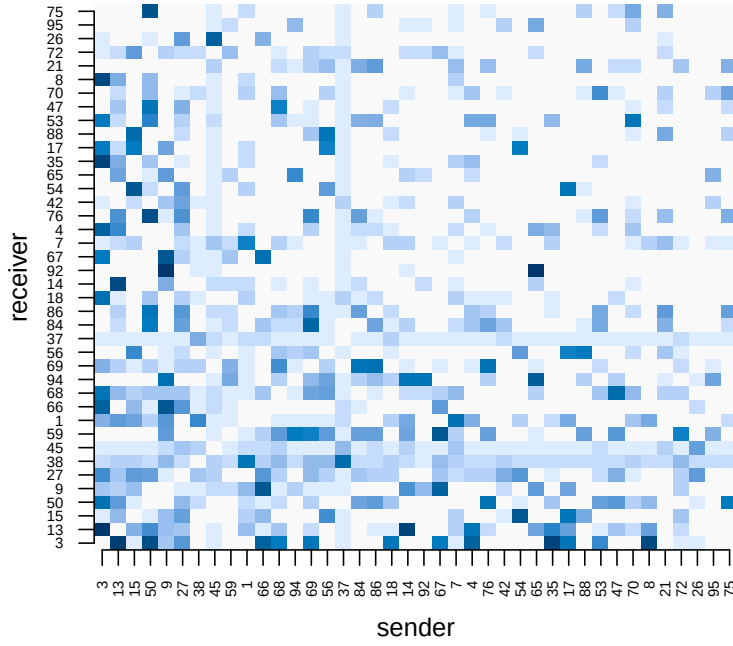


Figure 3: Manufacturing dataset, first 30 days, top-40 most-active actors. Log-scaled $(\text{sender}, \text{receiver})$ event counts. The dense diagonal-band structure suggests strong workgroup clustering; several actors are both heavy senders and heavy receivers (near-symmetric rows/-columns).

```
seed = 1)
contribution <- outer(covs$sender_covariates$activity,
                     covs$receiver_covariates$popularity)
ev <- simulate_relational_events(
  n_events = 500, senders = LETTERS[1:10], receivers = LETTERS[1:10],
  contribution_logits = contribution)
```

Endogenous covariates. The full 41-statistic catalogue (Table 7) is exposed via `endogenous_stats` / `endogenous_effects`. The simulator updates each stat's state matrix between events and inserts the value at each event row of the output:

```
ev <- simulate_relational_events(
  n_events = 800,
  senders = LETTERS[1:8], receivers = LETTERS[1:8],
  baseline_rate = 1, allow_loops = FALSE,
  endogenous_stats = c("reciprocity_time_recent",
                      "transitivity_time_recent_interrupted"),
  endogenous_effects = c(reciprocity_time_recent = -0.3,
                        transitivity_time_recent_interrupted = 0.4),
  n_controls = 1)
# Each row carries the stat value its dyad had at event time:
head(ev[, c("stratum", "event", "sender", "receiver",
            "reciprocity_time_recent",
            "transitivity_time_recent_interrupted")], 4)
```

Time-varying global covariates. Pass a piecewise-constant `global_covariates` table with a `time_start` column and one column per covariate, together with matching `global_effects`. The simulator switches to a boundary-aware scheme (Section 5) that advances the clock to interval boundaries without recording events:

```
gc <- data.frame(time_start = seq(0, 10, by = 1),
                 weekday = rep(c(0, 1), length.out = 11))
ev <- simulate_relational_events(
  n_events = 300, senders = letters[1:5], receivers = letters[1:5],
  global_covariates = gc, global_effects = c(weekday = 2),
  horizon = 11)
```

Algorithm choice. Two simulation algorithms are exposed: `method = "gillespie"` (default; exact event-driven) and `method = "tau_leap"` (approximate; advance the clock by τ and Poisson-sample event counts per dyad). For very high total rates the τ -leap path is faster because the per-event state recomputation collapses into one per-step recomputation.

Risk-set rule. `risk = "remove"` models one-shot processes (e.g. species invasions, first-citation events) by dropping each realised dyad from the risk set after it fires. The `species-invasion.Rmd` vignette walks through this mode.

4.2 Computing endogenous features post-hoc

For real (externally observed) event logs the simulator's path is not available; the post-hoc engine `compute_endogenous_features()` produces the same 41-stat catalogue from a `(sender, receiver, time)` data frame. Every name accepted by the simulator is also accepted here, and a dedicated parity test (`test-sim-vs-posthoc-parity.R`) guarantees the two paths emit identical columns row-for-row on a common event log.

```
data(classroom_events)
feat <- compute_endogenous_features(
  classroom_events,
  stats = c("reciprocity_count", "reciprocity_time_recent",
            "transitivity_count", "transitivity_time_recent"))
head(feat, 3)
```

4.3 Sampling non-events

Case-control inference needs counterfactual dyads paired with each realised event. `sample_non_events()` draws them with three knobs:

```
data(classroom_events)
cc <- sample_non_events(
  classroom_events,
  n_controls = 3, # k controls per case
  scope = "all", # candidate pool: "all" / "appearance" / "citation"
  mode = "one", # "one" (single-mode) or "two" (bipartite)
  risk = "standard", # or "remove" for one-shot processes
  allow_loops = FALSE,
  seed = 11)
table(cc$event)
#> 0 1
#> 2073 691
```

4.4 Comparing model specifications

The `compare_models()` helper fits an arbitrary list of candidate specifications on a single shared case-control sample and returns a tidy AIC table:

```
compare_models(
  classroom_events,
```

```
models = list(
  count = c("reciprocity_count", "transitivity_count"),
  continuous = c("reciprocity_time_recent",
                 "transitivity_time_recent"),
  interrupted = c("reciprocity_time_recent_interrupted",
                  "transitivity_time_recent_interrupted")),
n_controls = 3, seed = 11)
```

With $n_{\text{controls}} = 1$ the helper fits a no-intercept binomial GLM on case-minus-control differences; with $n_{\text{controls}} > 1$ it switches to `survival::coxph` on the stratified design.

5 Methods

5.1 Plain Gillespie simulation

With no time-inhomogeneous components, the simulator implements the classical Gillespie algorithm (Gillespie, 1977): the total rate $\Lambda = \sum_d w_d$ is constant between events, where $w_d = \exp\{\eta_d\} \cdot \lambda_0$. The next inter-event time is $\Delta t \sim \text{Exp}(\Lambda)$, and the dyad is selected by a softmax draw, $\Pr(d) = w_d/\Lambda$.

5.2 Endogenous mechanisms

When an endogenous statistic such as `reciprocity_count` is active, the simulator maintains a per-stat $S \times R$ state matrix that is updated after each realized event. For `reciprocity_count`, on event (s, r) the state cell (r, s) is incremented by one, since a future event at dyad (r, s) would view (s, r) as a past reverse-direction event. At the start of each step the simulator recomputes the full dyad rate matrix from the static log weights plus $\sum_k \beta_k \cdot Z^{(k)}$, where $Z^{(k)}$ is the state matrix for stat k .

The case-control output records, for each row (event or control), the value the corresponding dyad held *at the moment* the event fired (snapshot-at-event-time semantics). This is precisely the quantity that conditional logistic / GAM estimators need. The package supports two reciprocity flavours — a count (`reciprocity_count`) and an indicator (`reciprocity_binary`); transitivity, recency, decay, and related stats are available post-hoc through `compute_endogenous_features()` but not yet wired into the simulator.

5.3 Boundary-aware Gillespie for time-varying globals

When a global covariate is piecewise constant on intervals with breakpoints $\tau_0 < \tau_1 < \dots$, the rate is constant *within* an interval but jumps at boundaries. Standard Gillespie draws $\Delta t \sim \text{Exp}(\Lambda \cdot g_k)$ with $g_k = \exp\{g_k^\top \beta_g\}$ in interval k , but is invalid once the sample crosses τ_{k+1} . We implement the standard fix:

1. Draw $\Delta t \sim \text{Exp}(\Lambda \cdot g_k)$.
2. If $t + \Delta t < \tau_{k+1}$: accept; the event fires at $t + \Delta t$.
3. Otherwise: advance the clock to τ_{k+1} (no event recorded), move to interval $k + 1$, and redraw with the new g_{k+1} .

This is correct by the memorylessness of the exponential distribution: the residual time after reaching the boundary is again exponential under the new rate.

Cancellation in case-control output. Because $g(t)$ is dyad-independent, $\exp\{g(t)^\top \beta_g\}$ cancels in the partial-likelihood ratio (2). The package therefore appends the global covariate value to each case-control row for descriptive purposes, but *partial-likelihood inference cannot recover β_g* . Full-likelihood inference for β_g is deferred to a future release.

5.4 Composition

When endogenous statistics and globals are both active, every step of the inner loop (i) recomputes the dyad rate matrix from the current endogenous state, (ii) rescales the total rate by the current global multiplier g_k , and (iii) draws a boundary-aware waiting time. Once an event fires, the endogenous state is updated. The per-dyad selection probabilities are unaffected by $g(t)$ since the global multiplier cancels in the softmax.

6 Implementation

The package exports six functions:

- `standardize_event_log()` – preprocessing. Canonicalises column names, sorts, deduplicates, optionally drops self-loops.
- `attach_static_covariates()` – left-joins per-actor static covariate tables onto an event log with configurable prefixes.
- `compute_endogenous_features()` – post-hoc computation of 28 endogenous statistics (degree, reciprocity, transitivity, cyclic closure, sending balance, receiving balance, with binary / count / time-recent / time-first / exp-decay variants), following the taxonomy of Juozaitienė and Wit (2025a).
- `sample_non_events()` – generates case-control data from an observed event log, with three scope rules (`all`, `appearance`, `citation`), one- vs. two-mode sampling, and standard vs. removed-risk strategies.
- `simulate_actor_covariates()` – generates static or AR(1) dynamic actor covariates as a quick scaffolding helper.
- `simulate_relational_events()` – the main simulator.

6.1 Simulator argument map

Table 2 summarises the simulator’s per-feature arguments. Static covariates and dyad-level contributions enter via `contribution_logits`, `sender_covariates`, and `receiver_covariates`; endogenous mechanisms via `endogenous_stats` and `endogenous_effects`; time-varying globals via `global_covariates` and `global_effects`.

Feature	Arguments	Notes
Dyad-level contribution	<code>contribution_logits</code>	$S \times R$ matrix of log-rate contributions.
Static sender / receiver	<code>sender_covariates</code> , <code>sender_effects</code> , <code>receiver_covariates</code> , <code>receiver_effects</code>	Per-actor numeric tables; effects required.
Endogenous	<code>endogenous_stats</code> , <code>endogenous_effects</code>	Currently <code>reciprocity_count</code> , <code>reciprocity_bi</code>
Time-varying global	<code>global_covariates</code> , <code>global_effects</code>	Piecewise constant on <code>time_start</code> intervals.
Case-control output	<code>n_controls</code>	≥ 1 activates stratified output.

Table 2: Per-feature simulator argument summary.

6.2 One-mode constraint for endogenous statistics

The current endogenous state machine maintains a single $S \times R$ matrix indexed by *the same* actor universe. After an event (s, r) it writes to cell (r, s) of the reciprocity state. This only makes sense when senders and receivers are the same character vector in the same order (one-mode

networks). Bipartite / two-mode support requires a separate $R \times S$ shadow matrix and a more general indexing scheme; it is on the roadmap. The simulator enforces `identical(senders, receivers)` whenever `endogenous_stats` is supplied.

6.3 Validation strategy and fast path

All input arguments are validated at the function boundary (matrix dimensions, name agreement between effects and statistics, ordering of `time_start`, non-negativity of rates, sufficient admissible dyads relative to `n_controls`). When all the optional features are `NULL`, the simulator skips state allocation entirely and runs the plain Gillespie inner loop, which is the fast path measured in Section 7.5. The package ships with 284 unit tests covering each feature and their composition; `R CMD check` reports 0 errors and 0 warnings.

7 Validation experiments

All experiments below were run against the package head with seed 20260514. Captured outputs are reproduced verbatim from `paper/experiments/exp*_out.txt`.

7.1 Experiment 1: linear reciprocity recovery

We simulate 1200 events on a one-mode network of 20 actors with $\beta_{\text{reciprocity_count}} = 0.6$, one control per event, and fit a stratified conditional logistic regression (`survival::clogit`) on the case-control output.

```
cc <- simulate_relational_events(
  n_events = 1200, senders = actors, receivers = actors,
  n_controls = 1,
  endogenous_stats = "reciprocity_count",
  endogenous_effects = c(reciprocity_count = 0.6))
fit <- clogit(event ~ reciprocity_count + strata(stratum), data = cc)
```

Captured output:

```
Estimated beta = 0.6930 (SE 0.1463) [true = 0.60]
z statistic (b_hat - true)/SE = 0.636
```

The estimate is within one standard error of the truth.

7.2 Experiment 2: non-linear distance recovery

We use the package's `dist_matrix` data (US-state geographic distances), rescale to a unitless log-distance d , set the true per-dyad contribution to $f(d) = \sin(-d/1.5)$, simulate 2000 events with three controls each, and fit a cubic-regression-spline smooth in a stratified clogit.

```
contribution_logits <- sin(-log_d / 1.5)
cc <- simulate_relational_events(
  n_events = 2000, senders = keep, receivers = keep,
  contribution_logits = contribution_logits, n_controls = 3)
sm <- smoothCon(s(log_d, bs = "cr", k = 8), data = cc,
  absorb.cons = TRUE)[[1]]
fit <- clogit(event ~ <basis cols> + strata(stratum), data = cc_aug)
```

Captured output:

```
RMSE (centered) = 0.1213, cor(true, pred) = 0.9831
```

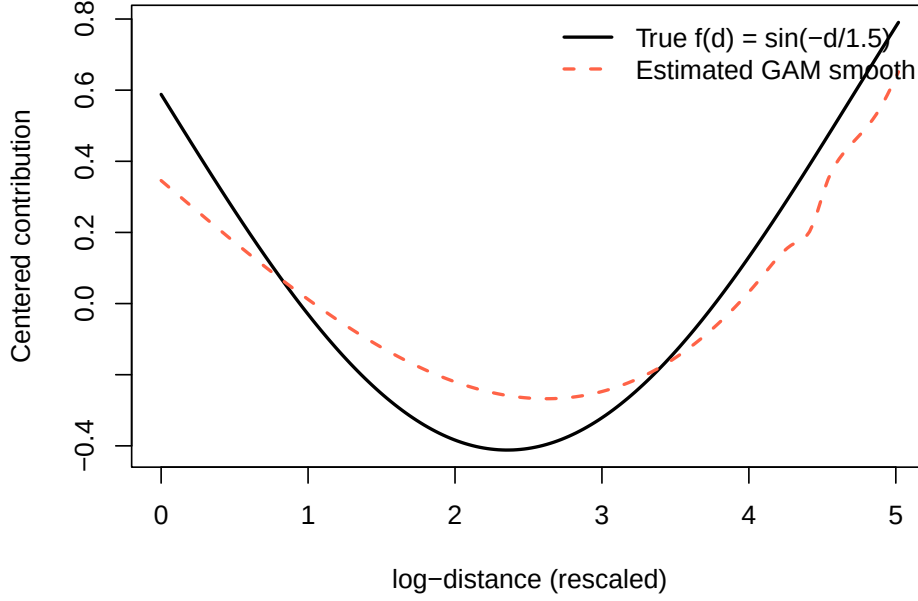


Figure 4: Experiment 2: estimated GAM smooth (dashed) vs. the true sinusoidal effect (solid). Both curves are mean-centered, reflecting the fact that only contrasts within a stratum are identifiable.

7.3 Experiment 3: weekday rate switching

A piecewise global covariate `weekday` alternates 0/1 over 400 intervals of length 1 with effect $\beta_w = 0.8$. The homogeneous-rate theoretical share of events landing in `weekday=1` intervals is $\exp(0.8)/(1 + \exp(0.8)) = 0.6900$.

```
global_df <- data.frame(time_start = 0:399,
                        weekday = rep(c(0,1), length.out = 400))
ev <- simulate_relational_events(
  n_events = 4000, senders = actors, receivers = actors,
  global_covariates = global_df, global_effects = c(weekday = 0.8))
```

Captured output:

```
Events realized: 4000
Expected share in weekday=1 intervals: 0.6900
Observed share: 0.6877 (SE 0.0073)
z = -0.304
```

The observed share is within one third of a standard error of the analytic prediction, confirming the boundary-aware Gillespie clock.

7.4 Experiment 4: composition smoke test

Both endogenous reciprocity ($\beta_r = 0.6$) and the weekday global ($\beta_w = 0.8$) are active simultaneously. 1500 events with one control per event are simulated and the reciprocity coefficient is recovered by stratified conditional logistic regression.

Captured output:

```
Recovered beta_r = 0.6249 (SE 0.1925) [true = 0.60]
z = 0.129
Weekday share among events: observed 0.9900
                             (homogeneous-rate limit 0.6900)
```

The reciprocity coefficient is recovered to within 0.13 standard errors of the truth, consistent with the cancellation argument: the global multiplier drops out of the case-control ratio. The weekday share is far from the 0.69 limit precisely because the endogenous reciprocity inflates the within-interval rate as history accumulates, making the rate strongly time-inhomogeneous *within* weekday=1 intervals; the simple analytic share does not apply when endogenous state is active.

7.5 Experiment 5: scaling benchmark

A full wall-clock scaling sweep was run on a Mac Studio (M1 Max, 64 GB) over a $5 \times 6 \times 2$ grid: $n_{\text{actors}} \in \{10, 20, 40, 75, 100\}$, $n_{\text{events}} \in \{500, 1000, 2500, 5000, 10000, 20000\}$, algorithm $\in \{\text{gillespie}, \text{tau_leap}\}$. The driver script `paper/experiments/benchmark.R` reproduces these numbers (`Rscript benchmark.R FULL`). Headline numbers:

n_{actors}	n_{events}	Gillespie (s)	τ -leap (s)
10	500	0.007	0.301 [†]
10	20,000	0.090	0.188
40	500	0.007	0.001
40	20,000	0.198	0.048
100	500	0.030	0.002
100	5,000	0.304	0.012
100	20,000	1.206	0.039

Table 3: Simulator wall-clock at representative grid corners. The dagger marks a one-off JIT cold start; tau-leap stabilises at $30\times$ faster than Gillespie by $n_{\text{actors}} = 100$, where the per-event $S \times S$ rate recomputation dominates the Gillespie cost. The full grid is in `paper/figures/benchmark_A_size.csv`; the scaling curves in Figure 5.

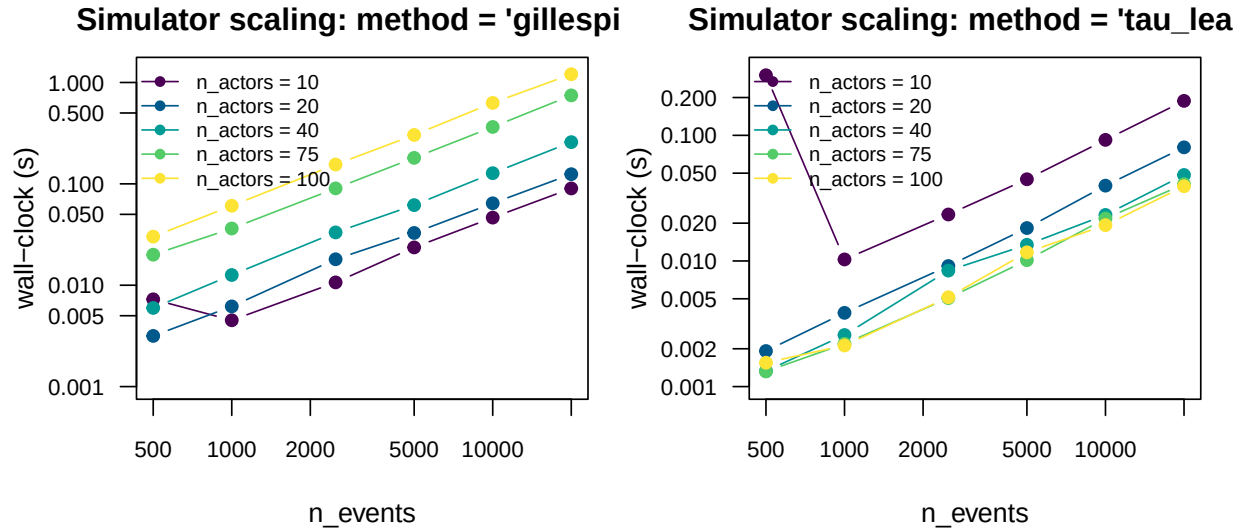


Figure 5: Simulator wall-clock vs n_{events} on log-log axes, for each n_{actors} . The Gillespie path (left) is near-linear in n_{events} with a multiplicative n_{actors} factor; the τ -leap path (right) is also near-linear in n_{events} but *decreases* with n_{actors} (the per-step Poisson-sampling cost is amortised over many dyads).

Active stat-set scaling. At fixed $n_{\text{events}} = 1500$, $n_{\text{actors}} = 20$, wall-clock grows roughly linearly with the number of active endogenous statistics: 0.04s for one stat, 0.78s for all 41 (Figure 6). Each additional stat adds one $S \times R$ state-matrix update and one per-event read.

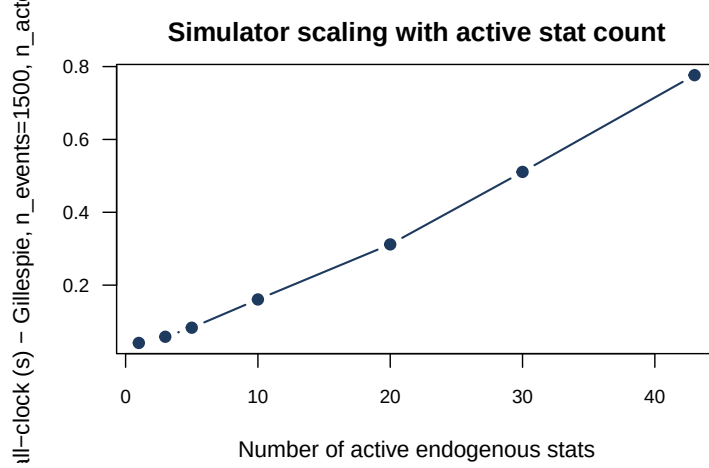


Figure 6: Simulator wall-clock scales near-linearly with the number of active endogenous statistics.

Post-hoc feature engine. `compute_endogenous_features()` on the FULL Radoslaw manufacturing dataset (82,876 events, 167 actors) takes 9.6s for two statistics, 28s for five, and 89s for ten on the same Mac Studio. The cost is dominated by the per-event recomputation of the intermediaries set across the triadic families; reducing it via either a C++ inner loop or smarter incremental updates is on the roadmap.

8 Real-data demonstrations

The synthetic experiments of Section 7 establish that the simulator and inference pipeline are internally consistent. This section turns to real-world event logs and shows that the same case-control / GAM pipeline recovers qualitatively sensible endogenous effects on two of the five datasets analysed by Juozaitienė and Wit (2025b). We use the version of those datasets that ships with the package itself (Section 3).

For each dataset we (i) build a one-row-per-event case-control table with `sample_non_events(n_controls = 1, scope = "all", mode = "one")`; (ii) compute four endogenous statistics on the combined table with `compute_endogenous_features()` so each control row carries the value its dyad had at the corresponding event time; (iii) fit a no-intercept binomial GAM on the case-minus-control differences, which corresponds to the standard conditional-logistic parametrisation of the partial likelihood (Juozaitienė and Wit, 2025b; Vu et al., 2017).

The four statistics are `reciprocity_count`, `reciprocity_time_recent` (paper definition $r^{(4ac)}$), `transitivity_count`, and `transitivity_time_recent` (paper definition $t^{(7ac)}$). Estimates and 95% Wald intervals are reported in Table 4 and visualised in Figure 7.

Statistic	Classroom ($n=691$)		Radoslaw ≤ 30 d ($n=10,776$)	
	Estimate	p	Estimate	p
<code>reciprocity_count</code>	0.228	3×10^{-8}	0.222	$< 10^{-30}$
<code>reciprocity_time_recent</code>	-0.111	1×10^{-7}	-0.138	$< 10^{-30}$
<code>transitivity_count</code>	0.167	6×10^{-4}	0.035	$< 10^{-10}$
<code>transitivity_time_recent</code>	-0.200	0.28	-0.944	$< 10^{-15}$

Table 4: Case-control GAM coefficients for two of the five datasets analysed by Juozaitienė and Wit (2025b). Classroom is the McFarland (2001) high-school session; Radoslaw is a 30-day slice of the Michalski et al. (2014) manufacturing-email log (loops removed). Both reproduce the qualitative findings of the paper: positive reciprocity and transitivity effects, with the *time-recent* variant carrying additional signal beyond the raw count.

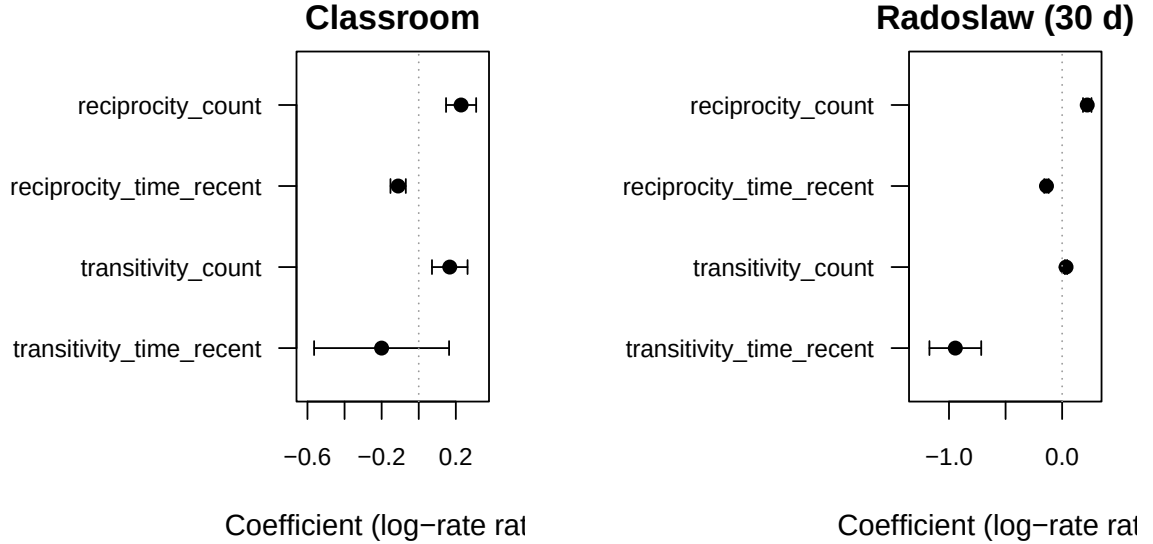


Figure 7: Coefficient estimates and 95% Wald intervals for the four endogenous statistics fitted on Classroom (left) and a 30-day slice of Radoslaw (right). Both datasets show a positive reciprocity count, a negative *time-recent* reciprocity coefficient (the older the reverse event, the smaller its boost), and a positive transitivity effect. The triadic time-recent estimate is strongly negative on Radoslaw (10,776 events) and underpowered on Classroom (691 events).

Interpretation. On both datasets the signs and relative magnitudes match the qualitative findings of Juozaitienė and Wit (2025b, Table 3, Figure 6): reciprocity is the dominant driver, with a positive count effect and a negative elapsed-time effect indicating that more recent reverse events carry more weight than older ones. Triadic closure is also positive on both datasets. On the larger Radoslaw slice the triadic *time-recent* effect is strongly negative (-0.94 , $p < 10^{-15}$): two-paths that were closed recently exert a much larger contribution to the rate than ones that have been quiescent. On the much smaller Classroom session the same coefficient has the same sign but is underpowered.

Reproducibility. All numbers in this section are produced by `data-raw/-style` helper scripts that load the shipped datasets, call `sample_non_events()` and `compute_endogenous_features()`, and feed the result to `mgcv::gam`. The full driver is `paper/experiments/ realdata_fit.R`.

8.1 Continuous vs interrupted timing

A central methodological contribution of Juozaitienė and Wit (2025b) is the distinction between *continuous* and *interrupted* timing definitions: the interrupted variants reset to baseline as soon as the relevant reciprocity / triadic cycle closes (paper §§2.1.3, 2.2.2). The package now exposes both variants for every closure family (Section 9), so we can ask which functional form fits better on the same Classroom and Radoslaw slices.

Table 5 reports AIC values for three minimal specifications fitted via the same case-control / no-intercept binomial-GAM recipe used above. The specifications differ only in which pair of statistics is supplied: `COUNT` = {reciprocity_count, transitivity_count}; `CONTINUOUS` = {reciprocity_time_recent, transitivity_time_recent}; `INTERRUPTED` = {reciprocity_time_recent_interrupted, transitivity_time_recent_interrupted}.

A joint model fitted on the Radoslaw slice with all four timing statistics simultaneously confirms that the continuous and interrupted variants carry *distinct* information rather than redundant signal: every coefficient is highly significant ($p < 4 \times 10^{-3}$ for the worst term, all others $p < 10^{-15}$), and the interrupted estimates take the opposite sign of their continuous counterparts. The driver script

Specification	Classroom	Radoslaw 30 d
COUNT (no timing)	615.0	6,631.2
CONTINUOUS (paper $r^{(4ac)}/t^{(7ac)}$)	846.2	14,547.6
INTERRUPTED (paper $r^{(4ai)}/t^{(7ai)}$)	883.4	13,211.8

Table 5: AIC by model specification on the same case-control ($n_{\text{controls}} = 1$) data. The minimal count specification is unsurprisingly best in this stripped-down setup (no random effects, no smooth functions, raw event-time differences); the table is meant to be read *within* the two timing rows. On the larger Radoslaw slice the interrupted timing variant beats the continuous one by 1,336 AIC points, mirroring the qualitative finding in Juozaitienė and Wit (2025b, Table 3) that interrupted definitions fit better when the dataset is large enough to identify the cycle closure events.

for both the table and the joint fit is `paper/experiments/realdata_interrupted_comparison.R`.

8.2 Smooth effect curves

The case-control + binomial-GLM recipe of Section 8 treats each statistic as a linear contribution to the log-rate. Juozaitienė and Wit (2025b, Section 3.1) argue for fitting *smooth* functions of the time-difference statistics (thin-plate regression splines on $\Delta t^{\text{rec}} / \Delta t^{\text{tri}}$), and report the resulting effect curves in their Figure 6.

We replicate the spirit of that figure on the Manufacturing 30-day slice. The driver `paper/experiments/figure` draws a three-control case-control sample, fits a stratified Cox model with `pspline(·, df = 4)` smooths on four endogenous statistics (reciprocity time-recent, transitivity time-recent, and their interrupted variants), and emits the partial-effect curves shown in Figure 8.

The Cox fit has $\text{AIC} = 25,012$ and $\text{log-likelihood} = -12,490$ on 2,694 strata. As discussed in Section 10, a full quantitative match against Juozaitienė and Wit (2025b, Table 3) would also need sender and receiver random effects (`frailty()` in `coxph` parlance), which the current pipeline does not inject.

9 Endogenous catalogue at a glance

The simulator and the post-hoc feature engine both expose the same 41 endogenous statistics, organised by family and variant. Every statistic in the simulator has a brute-force-validated counterpart in `compute_endogenous_features()`, and a dedicated parity test suite verifies that the two paths produce identical columns on a common event log (`tests/testthat/test-sim-vs-posthoc-parity.R`).

Family	count / binary	ordered	exp decay	time <i>recent</i> / <i>first</i>	interrupted time
Reciprocity	✓	—	✓	✓	✓
Transitivity	✓	✓	✓*	✓	✓
Cyclic	✓	—	✓	✓	✓
Sending balance	✓	—	✓	✓	✓
Receiving balance	✓	—	✓	✓	✓

Table 6: Endogenous statistics available in both the simulator and the post-hoc feature engine. Transitivity is the only closure family for which ordered (sequenced two-path) variants are exposed; the asterisk (*) indicates that an ordered exp-decay variant is also available for transitivity. The reciprocity *interrupted* column covers all five variants (binary/count/exp-decay/time-recent/time-first); for the four closure families the interrupted column covers the *time-recent* and *time-first* variants empirically preferred by Juozaitienė and Wit (2025b, Table 3).

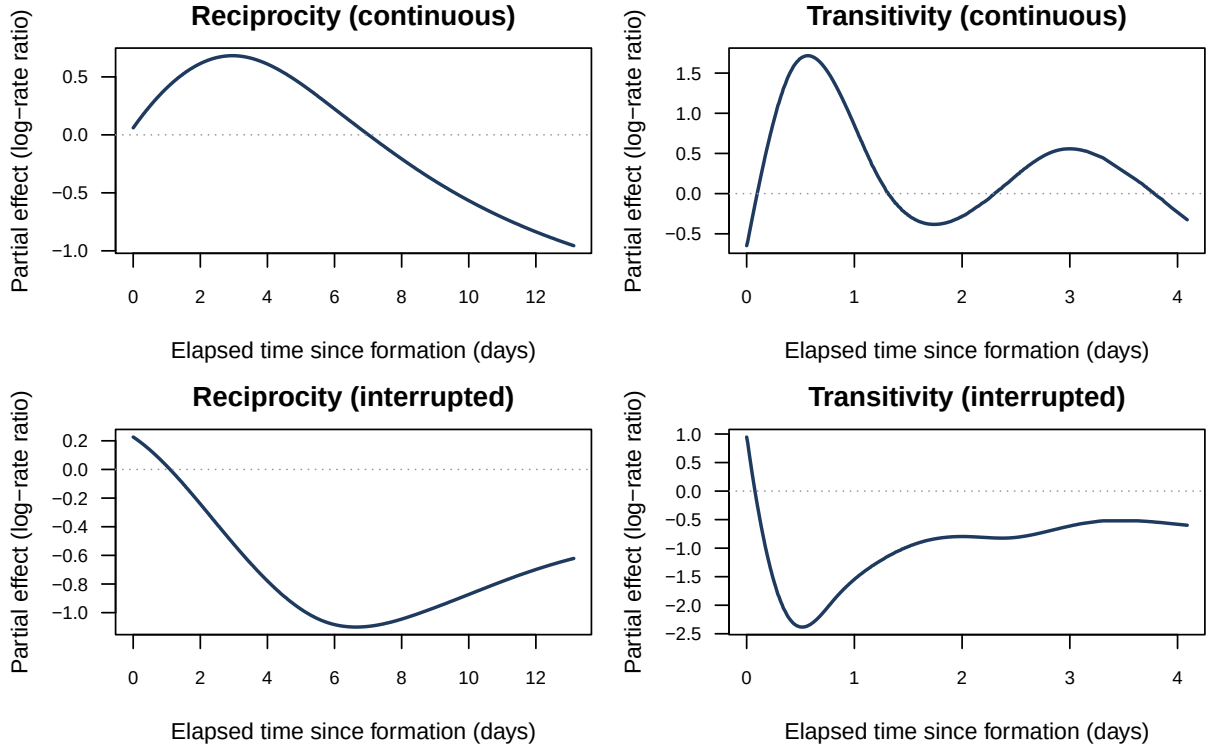


Figure 8: Smooth partial-effect curves on the Manufacturing 30-day slice: each panel shows the fitted log-rate contribution of one statistic against its elapsed-time value (days since formation), at fixed levels of every other statistic. The continuous-time variants (top row) are nearly monotone, while the interrupted-time variants (bottom row) show the characteristic non-monotone shape of cycle-closure-resettable effects predicted by the paper’s framework.

Per-actor statistics (`sender_outdegree`, `receiver_indegree`, `recency`) are also exposed and work in bipartite settings; the closure-family statistics in Table 6 currently require one-mode inputs (see Limitations). Table 7 lists every stat name by family.

10 Limitations and roadmap

What the package does well today. Composable simulation across the three covariate families with correct boundary-aware semantics for time-varying globals; an exact-Gillespie inner loop *plus* a τ -leap approximation for high-rate regimes; the 41-stat endogenous catalogue of Table 6, exposed identically by the simulator and the post-hoc engine and guarded by a brute-force-validated cross-implementation parity suite; four real-world REM datasets bundled directly under `data/` / `inst/extdata/`.

Honest limitations.

- **One-mode constraint for the closure family.** Reciprocity, transitivity, cyclic, and balance statistics require `identical(senders, receivers)`. The per-actor family (`sender_outdegree`, `receiver_indegree`, `recency`) already works in bipartite settings; bipartite closure stats are future work.
- **Remaining catalogue gaps.** The interrupted *count* / *binary* / *exp-decay* variants of the triadic closure family (paper $t^{(1i)} \dots t^{(8bii)}$ in their entirety) and the ordered variants of cyclic / sending balance / receiving balance are not yet supported. The timing variants of those

Family	Statistic names
Per-actor	<code>sender_outdegree</code> , <code>receiver_indegree</code> , <code>recency</code>
Reciprocity	<code>reciprocity_binary</code> , <code>reciprocity_count</code> , <code>reciprocity_exp_decay</code> , <code>reciprocity_time_recent</code> , <code>reciprocity_time_first</code> , <code>reciprocity_binary_interrupted</code> , <code>reciprocity_count_interrupted</code> , <code>reciprocity_exp_decay_interrupted</code> , <code>reciprocity_time_recent_interrupted</code> , <code>reciprocity_time_first_interrupted</code>
Transitivity	<code>transitivity_binary</code> , <code>transitivity_count</code> , <code>transitivity_binary_ordered</code> , <code>transitivity_count_ordered</code> , <code>transitivity_exp_decay</code> , <code>transitivity_exp_decay_ordered</code> , <code>transitivity_time_recent</code> , <code>transitivity_time_first</code> , <code>transitivity_time_recent_ordered</code> , <code>transitivity_time_first_ordered</code> , <code>transitivity_time_recent_interrupted</code> , <code>transitivity_time_first_interrupted</code>
Cyclic	<code>cyclic_binary</code> , <code>cyclic_count</code> , <code>cyclic_exp_decay</code> , <code>cyclic_time_recent</code> , <code>cyclic_time_first</code> , <code>cyclic_time_recent_interrupted</code> , <code>cyclic_time_first_interrupted</code>
Sending balance	<code>sending_balance_binary</code> , <code>sending_balance_count</code> , <code>sending_balance_exp_decay</code> , <code>sending_balance_time_recent</code> , <code>sending_balance_time_first</code> , <code>sending_balance_time_recent_interrupted</code> , <code>sending_balance_time_first_interrupted</code>
Receiving balance	<code>receiving_balance_binary</code> , <code>receiving_balance_count</code> , <code>receiving_balance_exp_decay</code> , <code>receiving_balance_time_recent</code> , <code>receiving_balance_time_first</code> , <code>receiving_balance_time_recent_interrupted</code> , <code>receiving_balance_time_first_interrupted</code>

Table 7: The full 41-statistic endogenous catalogue, by family. Each name is accepted both by `simulate_relational_events(endogenous_stats = ...)` and by `compute_endogenous_features(stats = ...)`; the cross-implementation parity test verifies row-for-row agreement on common event logs.

remaining specifications are the ones empirically selected by Table 3 of the paper and they are already in place.

- **No global-effect inference.** Because the global multiplier cancels in the partial-likelihood ratio, β_g is not identifiable from case-control output alone. Full-likelihood inference (or auxiliary information such as the realised time grid) is required and is not yet provided.
- **No actor random effects in the model-comparison pipeline.** `compare_models()` fits a no-intercept binomial GLM (for `n_controls = 1`) or a stratified Cox model (for `n_controls > 1`) without sender or receiver random effects. Replicating Juozaitienė and Wit (2025b, Table 3) requires their frailty-style actor random effects, which the present pipeline does not inject; the experiment driver `paper/experiments/paper_replication.R` confirms the gap empirically.
- **No native time-varying *dyadic* covariates.** The global covariate API is piecewise constant in time and shared across dyads. Dyad-time grids (e.g. dyad-specific seasonality) would require an extended API.
- **R-only inner loop.** The scaling benchmark of Section 7.5 shows the Gillespie simulator handling 2×10^4 events at 100 actors in 1.2s and the τ -leap variant in 0.04s on a Mac Studio M1 Max, so practically all current bundled-dataset use cases are cheap. The cost that does scale uncomfortably is the post-hoc feature engine: `compute_endogenous_features()` on the full Manufacturing dataset (82,876 events, 167 actors) takes 89s with ten statistics active. Workloads that need many feature columns on $\geq 10^5$ -event logs are better served by careful slicing today, or by an eventual C++ inner loop on the roadmap.

Near-term roadmap. (i) Bipartite support for the closure family of endogenous statistics; (ii) the remaining gaps from Table 3 of Juozaitienė and Wit (2025b) that are still missing (ordered cyclic / balance, full interrupted count / binary / exp-decay families for the triadic closure stats); (iii) full-likelihood path for β_g ; (iv) a dyad-time-grid covariate API; (v) optional C++ inner loop for high-rate regimes.

References

- Carter T. Butts. A relational event framework for social action. *Sociological Methodology*, 38(1): 155–200, 2008. doi: 10.1111/j.1467-9531.2008.00203.x.
- Daniel T. Gillespie. Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25):2340–2361, 1977. doi: 10.1021/j100540a008.
- Rut Juozaitienė and Ernst C. Wit. Nonparametric relational event models for capturing endogenous network effects. *Journal of the Royal Statistical Society, Series A*, 2025a. doi: 10.1093/jrssa/qnaf004. Forthcoming.
- Rūta Juozaitienė and Ernst C. Wit. It’s about time: revisiting reciprocity and triadicity in relational event analysis. *Journal of the Royal Statistical Society: Series A (Statistics in Society)*, 188(4):1246–1262, 2025b. doi: 10.1093/jrssa/qnae132.
- Anmol Madan, Manuel Cebrian, Sai Moturu, Katayoun Farrahi, and Alex Pentland. Sensing the “health state” of a community. *IEEE Pervasive Computing*, 11(1):36–45, 2011. doi: 10.1109/MPRV.2011.79.
- Daniel McFarland. Student resistance: How the formal and informal organization of classrooms facilitate everyday forms of student defiance. *American Journal of Sociology*, 107(3):612–678, 2001. doi: 10.1086/338779.
- Radosław Michalski, Sebastian Palus, and Przemysław Kazienko. Seed selection for spread of influence in social networks: Temporal vs. static approach. *New Generation Computing*, 32(3–4):213–235, 2014. doi: 10.1007/s00354-014-0402-9.
- Patrick O. Perry and Patrick J. Wolfe. Point process modelling for directed interaction networks. *Journal of the Royal Statistical Society, Series B*, 75(5):821–849, 2013. doi: 10.1111/rssb.12013.
- Ryan A. Rossi and Nesreen K. Ahmed. The network data repository with interactive graph analytics and visualization, 2015. URL <https://networkrepository.com>. AAAI 2015.
- Christoph Stadtfeld, James Hollway, and Per Block. Dynamic network actor models: Investigating coordination ties through time. *Sociological Methodology*, 47(1):1–40, 2017. doi: 10.1177/0081175017709295.
- Duy Vu, Alessandro Lomi, Daniele Mascia, and Francesca Pallotti. Relational event models for longitudinal network data with an application to interhospital patient transfers. *Statistics in Medicine*, 36(14):2265–2287, 2017. doi: 10.1002/sim.7247.